Rappresentazione di un heap

Materia	Algoritmi
Anno	Secondo Anno
Data	@October 15, 2024
Studiato	Done

La differenza tra la rappresentazione astratta di un heap e la sua effettiva rappresentazione fisica è notevole. Infatti, possiamo rappresentare l'heap come un **array** $A[1, ..., n]$, in cui:

- $A[1]$ è la **radice dell'heap**;
- $A[2i]$ è il **figlio sinistro dell'elemento di posto i;**
- $A[2i + 1]$ è il **figlio destro dell'elemento di posto i.**

Nota: tali considerazioni sono valide in virtù dell'indicizzazione $[1, ..., n]$. Tuttavia la maggior parte dei linguaggi di programmazione seguono l'indicizzazione $[0, ..., n - 1]$. Si ha allora che il figlio sinistro è $2i + 1$ e il figlio destro $2i$.

In questo modo quando devo eliminare un elemento, e quindi eliminare l'ultimo nodo, basta eliminare l'elemento finale dell'array. Questa posizione n prende il nome di **heap size**.

Naturalmente se un elemento è di posto i, possiamo semplicemente ottenere la posizione del nodo padre calcolando $\lfloor \frac{i}{2} \rfloor$.

```
left(i):
    return 2i

right(i):
    return 2i+1
```

```
parent(i):
    return floor(i/2)
```

La moltiplicazione per 2, inoltre, coincide con uno shift di 1 bit a sinistra. Analogamente, spostando a destra di 1 bit dividiamo per 2. Quindi possiamo riformulare:

```
left(i):
    return (i << 1)

right(i):
    return (i << 1) | 1

parent(i):
    return (i >> 1)
```

Il risultato è migliore perché l'operazione di shift prende solo un ciclo di clock, mentre la moltiplicazione più cicli di clock.

Traduciamo ora ogni procedura in funzione dell'implementazione ad array.

Heapify

```
heapify(A, i):
    l = left(i)
    r = right(i)
    min = i
    if l <= heapsize and A[l] < A[min] then
        min = l
    if r <= heapsize and A[r] < A[min] then
        min = r
    if min != i then
        swap(A, i, min)
        heapify(A, min)
```

Extract-min

```
extract_min(A)
    swap(A, 1, heapsize)
    heapsize = heapsize - 1
    heapify(A, 1)
    return A[heapsize+1]
```

Insert

L'inserimento del nodo avviene sempre al livello più basso, e si fa risalire il nodo fino a parent.

```
insert(A,k):
    heapsize = heapsize + 1
    A[heapsize] = k
    i = heapsize
    p = parent(i)
    while p != 0 and A[p] > A[i] do
        swap(A, i, p)
        i = p
        p = parent(i)
```

Rendere heap un array

Nel momento in cui inizializziamo un array, possiamo procedere prendendo la nostra lista di elementi e applicando la procedura di `insert` descritta precedentemente. Di conseguenza, la procedura di inizializzazione di un heap ha complessità $O(n \log n)$.

Tuttavia, possiamo anche prendere la lista di elementi, inserirli normalmente all'interno di un array (quindi senza particolare ordine), e ragionare sul fatto che **ogni foglia è un heap**, quindi **possiamo applicare la procedura** `heapify` **dal basso verso l'alto**. Infatti, `heapify` è applicabile **solamente se i figli del nodo padre sono degli heap**.

Dal basso verso l'alto, il primo nodo che non è foglia è di posizione $\lfloor \frac{i}{2} \rfloor$, in quanto se abbiamo n nodi, $\lceil \frac{n}{2} \rceil$ nodi sono foglie. La procedura di costruzione di heap è la

seguente:

```
build_min_heap(A, n):
    for i = floor(n/2) to 1 do
        heapify(A, i)
    heapsize = n
```

Si può dimostrare matematicamente che la complessità di questa procedura è $O(n)$.

HeapSort

La struttura heap può essere utilizzata per ordinare gli elementi. Tale ordinamento prende il nome di **HeapSort**.

Allora consideriamo il seguente array:

$$[14, 15, 5, 7, 9, 8, 3, 2, 6, 12, 11]$$

Chiamiamo la procedura `build_max_heap` per costruire un max-heap. Otteniamo il seguente array:

$$[15, 14, 8, 7, 12, 5, 3, 2, 6, 9, 11]$$

Ottenuto in tempo $O(n)$.
Ora applichiamo una tecnica simile al **selection sort**: prendiamo il massimo elemento, e lo poniamo alla fine. Per fare ciò, basta prendere il primo elemento, ossia la **radice del max heap**, e lo scambiamo con l'ultimo elemento dell'array.

Quindi, ad ogni iterazione, utilizziamo la procedura `extract_max` e poniamo l'elemento massimo dell'array in ultima posizione. Questa procedura si ripete $n - 1$ volte.

```
heapsort(A, n):
    build_max_heap(A, n)
```

```
    for i = 1 to n - 1 do
        extract_max(A)
```

La complessità dell'algoritmo è $O(n \log n)$, ossia la stessa complessità del merge sort. Quindi è un algoritmo **ottimo**, perché meglio di così non si può fare.

Il vantaggio di questo algoritmo è che è **in loco**, perché spostiamo gli elementi all'interno dell'array. Quindi abbiamo un algoritmo ottimo e in loco.

La struttura max-heap in teoria nasce per implementare la coda con priorità, ed in particolare elementi con la stessa chiave vengono gestiti in maniera FIFO. Allora l'algoritmo di HeapSort è **stabile**, perché mantiene l'ordinamento relativo degli elementi con stessa chiave.